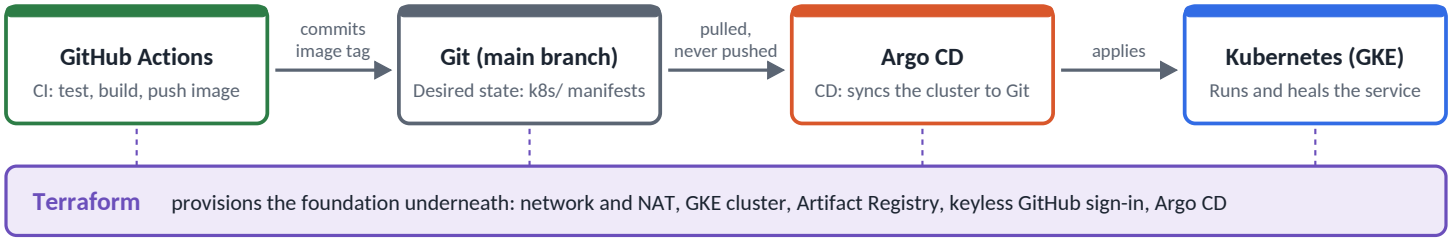
Why this stack

Kubernetes, Terraform, Argo CD and GitHub Actions for the hello-api delivery pipeline

gke-gitops-demo
github.com/whyatb/gke-gitops-demo

The service itself is deliberately small; the assignment is about the delivery platform around it. For software that regulated financial institutions depend on, that platform has to make every change **reviewable, reproducible and reversible**, and give no component more access than its job needs. The stack splits delivery into layers, each owned by one tool. The boundaries between them are what produce those guarantees.



Kubernetes on GKE

Runs the service and keeps it running

- **Declarative and self-healing.** We declare the outcome (2 replicas, probes, limits); failed pods restart and GKE repairs nodes.
- **Safe releases built in.** Rolling updates never drop below 2 ready pods; readiness probes keep traffic off a pod until it can serve.
- **Security as config.** Non-root, read-only containers under the *restricted* profile; a NetworkPolicy blocks all outbound traffic.
- **Portable.** The same manifests run on any conformant cluster, so the workload isn't tied to one cloud's proprietary platform.

Terraform

Builds the platform underneath, as code

- **Reproducible.** One `terraform apply` creates all 24 resources, network to Argo CD; built, used and destroyed cleanly.
- **Reviewable.** Changes are code diffs with a plan preview before anything happens: what change control needs.
- **Accountable.** Versioned state records exactly what exists, so a destroy leaves no forgotten, still-billing resources behind.
- **Least privilege, written down.** Each service account and its single role is defined in code, not clicked together in a console.

GitHub Actions

Continuous integration: test, build, publish

- **Tests on every change.** Pushes and pull requests run the tests; only *main* can publish an image, and Google enforces it.
- **Keyless.** Signs in to Google Cloud with short-lived OIDC tokens. No service account key exists to leak or rotate.
- **Traceable.** Images are tagged with the commit SHA and tags can't be overwritten; each pod reports its commit.
- **Deliberately limited.** It writes to Git and the registry only. It has no access to the cluster.

Argo CD

Continuous delivery: Git is the source of truth

- **Pull, not push.** It runs inside the cluster and fetches from Git, so no outside system holds credentials to the cluster.
- **Every deployment is a commit.** Who, what and when are in Git history; rollback is `git revert`.
- **Drift is corrected.** Manual changes made in the cluster are reverted automatically, so what runs is always what was reviewed.
- **Scoped and extensible.** A project confines the app to this repo and namespace; new services are added by commit (app of apps).

Supporting choices

Workload Identity Federation	Keyless CI; only this repository's main branch is accepted.
Artifact Registry	Immutable tags, and every pushed image is scanned for known vulnerabilities.
Private nodes + Cloud NAT	Nodes have no public IPs; they reach the internet outbound only, through NAT.
Kustomize	Plain YAML, no templating; a release changes exactly one line.
Go + distroless image	16 MB, no shell, no third-party packages: little to attack or patch.

Alternatives considered

- **CI deploys with `kubectl` (push model).** Fewer parts, but CI would need cluster-admin credentials, and the cluster could drift from Git unnoticed.
- **Console setup or scripts instead of Terraform.** Faster once, but not reproducible, not reviewable, and easy to leave billable resources behind.
- **Cloud Run instead of Kubernetes.** Simpler for one stateless service, but gives up the scheduling, policy controls and portability a growing platform needs.

Proven end to end. Built from an empty Google Cloud project (24 resources), deployed through the whole pipeline (commit 792ad34 answered from the public load balancer), then torn down to zero billable resources. The exact procedure is in `RUNBOOK.md`.